

Directed Graph Hashing

Caleb Helbling*

Abstract. In this paper we present various methodologies and algorithmic implementations for hashing node-labeled directed graphs. We distinguish between hashing graphs based on their global structure and hashing based on local observations obtained by traversing the graph. Our approaches are careful to take into account any symmetry in the graph, so that nodes that are identical modulo automorphism are assigned equal hashes. Structurally hashing trees has seen widespread use in blockchain, source code version control, and web applications. Despite the popularity of tree hashing, directed graph hashing remains unstudied in the literature.

1 Introduction Hashing functions are fundamental building blocks in the modern software stack that have seen widespread use in data storage and retrieval, digital fingerprinting, checksum and cryptographic applications. Traditionally, hash functions take as input bit strings of arbitrary length and return a bit string of a fixed size $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n$. Good hashing functions should be fast to compute, minimize the chance of duplication of output values, and produce large changes in output for small changes in input. Many bit string hashing functions have been created over the years, such as MD5 [17], SHA-2 [14], and SHA-3 [15].

Algorithm 1.1 Recursive Merkle-tree Hashing of Ordered Binary Tree

$$\text{mh}(n) = \begin{cases} \mathcal{H}(n.\text{label}) & n \text{ is a leaf} \\ \mathcal{H}(n.\text{label} \circ \text{mh}(n.\text{left}) \circ \text{mh}(n.\text{right})) & \text{otherwise} \end{cases}$$

Hashing functions can be extended to trees via a simple extension of Merkle hashing [11]. In Merkle tree hashing, the hash of a node is computed by hashing the concatenation of the node's label with the recursive hash of the node's children. Algorithm 1.1 gives a simple recursive implementation of Merkle tree hashing for ordered binary trees. This particular variation of Merkle hashing has seen use in many applications, including cryptocurrency [13], version control systems [9], and distributed hypertext document storage systems [1].

Extending hashing to node-labeled directed graphs is non-trivial due to the presence of cycles. Indeed, the design space of algorithm design for hashing digraphs is surprisingly broad and depends on use case objectives. In this paper we will focus on the following approaches, which we think are the most useful for practical applications:

- **Global Hashing** (Section 3) - for use cases where the *global* structure of the digraph is of importance. A correct algorithm should produce equal hashes if and only if two graphs are isomorphic. Nodes in the graph should hash to the same value if and only if they come from isomorphic graphs and they are identical modulo automorphism.
- **Local Hashing** (Section 4) - for use cases where observers can only observe *local* information by traversing from a node to a neighbor on an outgoing edge. There are several formulations of local hashing, but in this paper we will discuss an algorithm that produces equal hashes for nodes if and only if the (potentially infinite) tree expansions from outward exploration are isomorphic. Local hashing has many commonalities with bisimulation and co-inductive representations of finite directed graphs [18, 16].
- **Ordered Outgoing Edges** (Section 5) - in many practical applications the outgoing edges from a node are ordered. We can use this additional structure to simplify and improve the runtime performance of our hashing algorithms.

*Draper (chelbling@draper.com).

- **Merkle Graph Hashing** (Section 6) - for use cases where we want graphs to be hashed recursively, or graphs are structures that are built up over time. In this approach we hash strongly connected components individually using either global or local hashing, then recurse over the acyclic condensation graph induced by the strongly connected components.

One common theme throughout this paper is reliance on conversion of graphs to *canonical form*. If two graphs (or nodes within) can be converted to a canonical form, we can serialize the canonical form and pass it through an ordinary string function.

We should additionally take care to distinguish between the hash of an entire graph vs the hash of a specific node within a graph. Our general strategy for computing the hash of a specific node is to combine the overall graph hash with the node's position in canonical order. Care must be taken to ensure that our index accounts for any symmetry present in the graph (ie, two nodes may be identical modulo automorphism).

2 Background We now give some brief graph theory definitions required for this paper, which a more general audience interested in hashing may not be informed about.

An *automorphism* of some graph $G = (V, E, L)$ is defined as a permutation σ of vertices such that $(u, v) \in E$ if and only if $(\sigma(u), \sigma(v)) \in E$. Here L is a labeling function that maps vertices to strings. Informally, the automorphism group gives all of the ways that G is symmetrical.

$$\text{Aut}(G) = \{\sigma : V \rightarrow V \mid (u, v) \in E \leftrightarrow (\sigma(u), \sigma(v)) \in E\}$$

A *vertex orbit* is the equivalence class of vertices of a graph under the action of automorphisms. Informally, the orbit of a vertex v can be thought of as the set of vertices that are symmetrically identical to v . Figure 2.1 gives an example of how orbits can be used to partition a graph.

$$\text{Orb}(v, G) = \{u \in V \mid \exists \sigma \in \text{Aut}(G) \text{ such that } \sigma(v) = u\}$$

Canonical labeling is the process of ordering graph vertices such that isomorphic graphs canonize to the same value. Note that canonical labeling algorithms cannot distinguish between different graph automorphisms. Algorithms that can compute canonical labels are widely used in practice for graph isomorphism testing. Some examples of canonical labeling algorithms include nauty [10], bliss [7], and saucy [5].

The recursive Merkle-style algorithm will utilize the analysis of cycles within a directed graph. A *strongly connected component* of a graph is a maximal set of vertices such that there exists a path in the graph between any two vertices within the strongly connected component. A *condensation graph* is formed by contracting strongly connected components into single nodes. We will exploit the acyclicity of the condensation graph to construct our

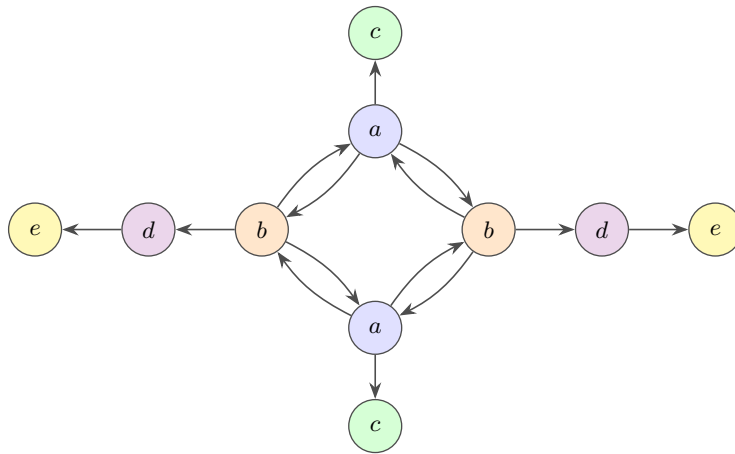


Figure 2.1: This directed graph contains five orbits, identified by the letters a , b , c , d and e and by the node colours. Nodes within the same orbit are structurally identical due to the symmetry of the directed graph: the drawing is invariant under a rotation by 180° , which maps every node onto its orbit partner.

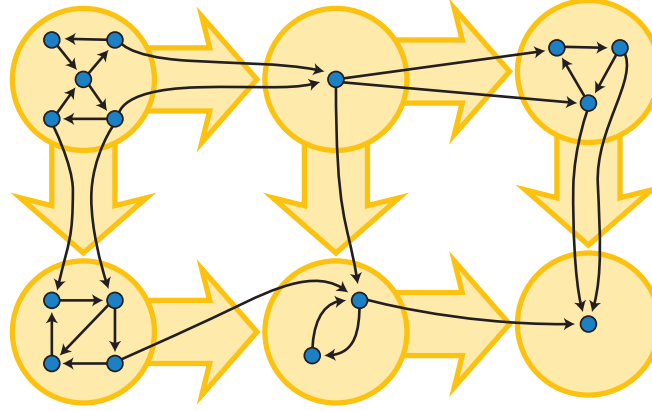


Figure 2.2: Each yellow node contains a strongly connected component of a graph. When each strongly connected component is contracted into a single node, the yellow condensation graph is formed. Since the condensation graph is acyclic, it can be recursed over. Public domain figure courtesy of David Eppstein.²

recursive algorithm. Figure 2.2 shows an example of strongly connected components and the induced condensation graph.

3 Global Graph Hashing In global graph hashing, we would like our computed hash values to be unique modulo graph isomorphism. To be more precise, the hash of a graph $\mathcal{H}_G$ has the property $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$. To achieve this correctness properties, we build on top of the well known string hashing functions and canonical labeling algorithms. We begin by canonizing the input digraph and constructing an adjacency list sorted in canonical order. The node labels (in canonical order) and the canonized adjacency list are then encoded as strings and hashed using a string hashing function $\mathcal{H}$. Algorithm 3.1 gives an implementation of this approach.

Note that since we use generic graph canonization and orbit detection algorithms, our approach does not run in polynomial time. Indeed, graph hashing can trivially be use used to decide graph isomorphism, and as a consequence is **GI-Hard**. Since graph isomorphism is not thought to have a polynomial time algorithm, this means graph hashing likely does not have a polynomial time implementation.

THEOREM 3.1 (Global Hash Proof of Correctness). *Assuming that the string hashing function $\mathcal{H}$ is injective, $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ if and only if $G_1 \cong G_2$.*

Proof. ($\Rightarrow$) If $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$, then the canonized adjacency list/vertex label lists are identical. If these representations of G_1 and G_2 are identical, then G_1 and G_2 have identical canonical form G_3 . This means that $G_1 \cong G_3$ and $G_2 \cong G_3$, which implies that $G_1 \cong G_2$ by transitivity.

($\Leftarrow$) Let G_3 be the output graph of canonizing G_1 , and G_4 be the output of canonizing graph G_2 . Since $G_1 \cong G_2$, we conclude that $G_3 = G_4$ since isomorphic graphs canonize to the same graph. This implies that

²https://commons.wikimedia.org/wiki/File:Graph_Condensation.svg

Algorithm 3.1 Global Digraph Hash

```

1: function  $\mathcal{H}_G(G=(V,E,L))$ 
2:                                      $\triangleright V$  is the set of vertices,  $E$  is the set of edges,  $L$  is the node labeling function
3:    $\rho \leftarrow \text{CANONIZE}(G)$   $\triangleright \text{CANONIZE}$  returns a mapping from vertices to their location in the canonical order
4:    $\text{adj\_list} \leftarrow \text{SORT}([( \rho(u), \rho(v)) \mid (u,v) \in E])$ 
5:    $\text{labels} \leftarrow [L(\rho^{-1}(n)) \text{ for } n \text{ in } 0..|V|]$ 
6:   return  $\mathcal{H}(\text{TO\_STR}((\text{labels}, \text{adj\_list})))$   $\triangleright$  Here  $\mathcal{H}$  is a string hashing function such as SHA-3
7: end function
```

Algorithm 3.2 Global Digraph Node Hash

```
1: function  $\mathcal{H}_{\mathcal{N}}(v, G)$ 
2:    $\rho \leftarrow \text{CANONIZE}(G)$ 
3:    $g\_hash \leftarrow \mathcal{H}_G(G)$ 
4:    $orbit\_idx \leftarrow \min(\text{Orb}(\rho(v), \rho(G)))$ 
5:   return  $\mathcal{H}(\text{TO\_STR}((orbit\_idx, g\_hash)))$ 
6: end function
```

the sorted adjacency lists and ordered vertex label lists are equivalent, so the string representation is equivalent. Therefore the string hash output are equivalent as well, which shows that $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$.

The algorithm for hashing nodes in a graph builds on the algorithm for hashing graphs. First, the graph is canonized and the hash of the entire canonical graph is computed using Algorithm 3.1. Then we compute the orbit index of the input vertex, which is defined to be the canonical index of the equivalence class representative element, where the equivalence relation in this case is the vertex orbit. The orbit index and the graph hash are then combined and passed through a string hashing function. Algorithm 3.2 gives an implementation of this approach.

THEOREM 3.2 (Global Node Hash Proof of Correctness). *Nodes should hash to the same value if and only if they come from isomorphic graphs and they are identical modulo vertex orbits. To be precise, $\mathcal{H}_{\mathcal{N}}(v_1, G_1) = \mathcal{H}_{\mathcal{N}}(v_2, G_2)$ if and only if $G_1 \cong G_2$, $\exists \phi.v_2 \in \text{Orb}(\phi(v_1), G_2)$, and $\exists \phi.v_1 \in \text{Orb}(\phi^{-1}(v_2), G_1)$ where $\phi : V_1 \rightarrow V_2$ is an isomorphism between G_1 and G_2 .*

Proof. ($\Rightarrow$) If $\mathcal{H}_{\mathcal{N}}(v_1, G_1) = \mathcal{H}_{\mathcal{N}}(v_2, G_2)$, then the orbit index of v_1 in $\mathcal{H}_{\mathcal{N}}(v_1, G_1)$ is equal to the orbit index of v_2 in $\mathcal{H}_{\mathcal{N}}(v_2, G_2)$, and $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ by the injectivity of string hashing. Then by Theorem 3.1, $G_1 \cong G_2$.

Let ρ_1 be the canonization of G_1 and ρ_2 be the canonization of G_2 . Then $\rho_1(G_1) = \rho_2(G_2)$ since $G_1 \cong G_2$. Note that ρ_1 and ρ_2 are both isomorphisms. Then let $\phi = \rho_2^{-1} \circ \rho_1$ and $\phi^{-1} = \rho_1^{-1} \circ \rho_2$. Applying this definition of ϕ , we wish to show that $v_2 \in \text{Orb}(\rho_2^{-1}(\rho_1(v_1)), G_2)$ and $v_1 \in \text{Orb}(\rho_1^{-1}(\rho_2(v_2)), G_1)$. Applying the isomorphisms ρ_2 and ρ_1 to these sets respectively, this is equivalent to showing that $\rho_2(v_2) \in \text{Orb}(\rho_1(v_1), \rho_2(G_2))$ and $\rho_1(v_1) \in \text{Orb}(\rho_2(v_2), \rho_1(G_1))$.

Recall that the orbit index of v_1 in $\mathcal{H}_{\mathcal{N}}(v_1, G_1)$ is equal to the orbit index of v_2 in $\mathcal{H}_{\mathcal{N}}(v_2, G_2)$. This implies that the two orbit sets are equal $\text{Orb}(\rho_1(v_1), \rho_1(G_1)) = \text{Orb}(\rho_2(v_2), \rho_2(G_2))$ since they have equal representative elements. Then since $\rho_1(G_1) = \rho_2(G_2)$, $\text{Orb}(\rho_1(v_1), \rho_1(G_1)) = \text{Orb}(\rho_2(v_2), \rho_2(G_2)) = \text{Orb}(\rho_1(v_1), \rho_2(G_2)) = \text{Orb}(\rho_2(v_2), \rho_1(G_1))$. Then trivially $\rho_1(v_1) \in \text{Orb}(\rho_1(v_1), \rho_1(G_1))$ and $\rho_2(v_2) \in \text{Orb}(\rho_2(v_2), \rho_2(G_2))$. Following the chain of equalities and making the appropriate substitutions, then $\rho_2(v_2) \in \text{Orb}(\rho_1(v_1), \rho_2(G_2))$ and $\rho_1(v_1) \in \text{Orb}(\rho_2(v_2), \rho_1(G_1))$.

($\Leftarrow$) If $G_1 \cong G_2$, then $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$ by Theorem 3.1. Then there exists some isomorphism ϕ and ϕ' such that $v_2 \in \text{Orb}(\phi(v_1), G_2)$ and $v_1 \in \text{Orb}(\phi'^{-1}(v_2), G_1)$. Let ρ_1 be the canonization of G_1 and ρ_2 be the canonization of G_2 . Applying these isomorphisms, we have $\rho_2(v_2) \in \text{Orb}(\rho_2(\phi(v_1)), \rho_2(G_2))$ and $\rho_1(v_1) \in \text{Orb}(\rho_1(\phi'^{-1}(v_2)), \rho_1(G_1))$. Let $f = \rho_1 \circ \phi^{-1} \circ \rho_2^{-1}$ and $f' = \rho_2 \circ \phi' \circ \rho_1^{-1}$ be automorphisms over the canonized graph.

Then by definition of orbit, $\text{Orb}(\rho_2(\phi(v_1)), \rho_2(G_2)) = \text{Orb}(f(\rho_2(\phi(v_1))), \rho_2(G_2))$ and $\text{Orb}(\rho_1(\phi'^{-1}(v_2)), \rho_1(G_1)) = \text{Orb}(f'(\rho_1(\phi'^{-1}(v_2))), \rho_1(G_1))$. Substituting in the definitions of f and f' , we have $\text{Orb}(\rho_2(\phi(v_1)), \rho_2(G_2)) = \text{Orb}(\rho_1(v_1), \rho_2(G_2))$ and $\text{Orb}(\rho_1(\phi'^{-1}(v_2)), \rho_1(G_1)) = \text{Orb}(\rho_2(v_2), \rho_1(G_1))$. This shows that $\rho_2(v_2) \in \text{Orb}(\rho_1(v_1), \rho_2(G_2))$ and $\rho_1(v_1) \in \text{Orb}(\rho_2(v_2), \rho_1(G_1))$. Since $G_1 \cong G_2$, $\rho_1(G_1) = \rho_2(G_2)$ so $\rho_2(v_2) \in \text{Orb}(\rho_1(v_1), \rho_1(G_1))$ and $\rho_1(v_1) \in \text{Orb}(\rho_2(v_2), \rho_2(G_2))$. This suffices to show that the orbit indices are equal in the computations of $\mathcal{H}_{\mathcal{N}}(v_1, G_1)$ and $\mathcal{H}_{\mathcal{N}}(v_2, G_2)$.

Since the orbit indices are equal and $\mathcal{H}_G(G_1) = \mathcal{H}_G(G_2)$, then $\mathcal{H}_{\mathcal{N}}(v_1, G_1) = \mathcal{H}_{\mathcal{N}}(v_2, G_2)$.

4 Local Graph Hashing In comparison to the global hashing strategy which depends on the global structure of the graph, the local hashing strategy depends only on properties we can *observe* by traversing the digraph forward along its edges. To make this concept precise, we say that the hash of two nodes in a digraph should be equal if and only if their unfoldings are isomorphic. An unfolding is the formed by selecting a starting node, placing it at the root of the tree, then adding neighboring nodes as children, and expanding recursively.

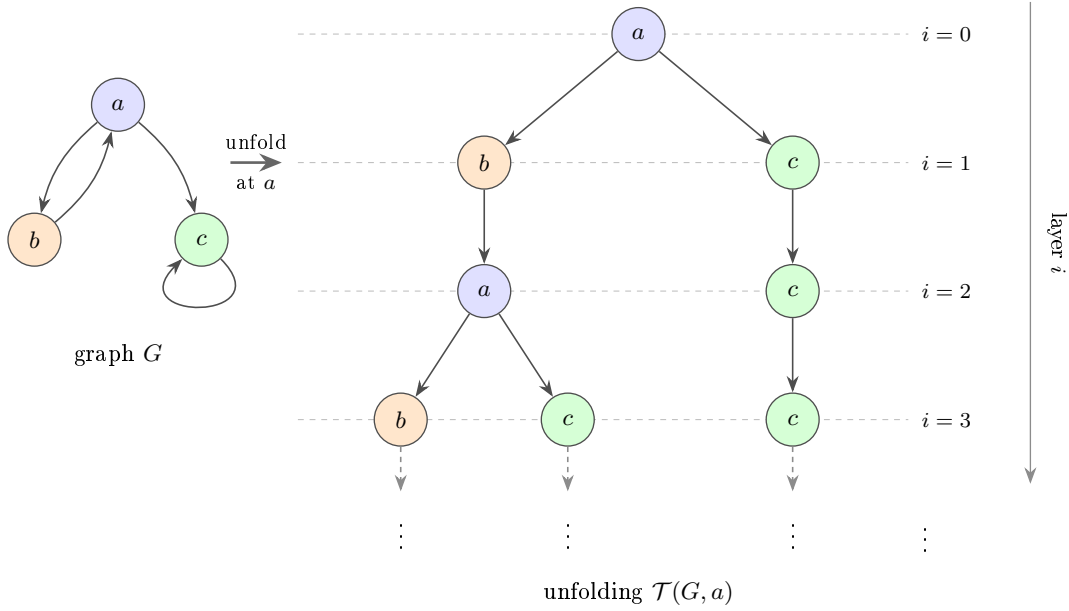


Figure 4.1: Unfolding of a directed graph with cycles. Starting from the root a , every vertex is replaced by one copy per path reaching it, so the cycle $a \rightarrow b \rightarrow a$ and the self-loop $c \rightarrow c$ are unrolled into an infinite tree; only the layers $i = 0, \dots, 3$ are drawn. Each tree node at layer i corresponds to a path of length i from a in G ; colours indicate the originating vertex of G . The color $\lambda_i(a)$ is equivalent to the hash of the unfolding finite Merkle-tree up to layer i .

This forms a (potentially infinite) tree induced by the structure of the underlying digraph.

Our strategy for local graph hashing uses a variation of an algorithm to compute color refinement. In color refinement, nodes are repeatedly colored and propagated over multiple iterations until a stable coloring is achieved. To be more precise, the initial coloring λ_0 of the nodes is defined to be the hash of the node's labels:

$$(4.1) \quad \lambda_0(u) = \mathcal{H}(L(u))$$

We then compute colorings over multiple iterations using the following equation (note that $\{\{\dots\}\}$ is a multiset, and that a multiset can be converted to a string by sorting its members):

$$(4.2) \quad \lambda_{i+1}(u) = \mathcal{H}(\text{TO_STR}((L(u), \{\{\lambda_i(v) \mid (u, v) \in E\}\})))$$

The color of a node in round i is exactly the Merkle-hash of an unfolding tree with a maximum depth of $i + 1$ layers. Note here that our unfolded finite tree is being built bottom-up, with a new layer added to the tree every iteration. Due to the bottom-up construction, the hashing process maximally shares computation, making this a dynamic programming approach. Note that we differ here from classic color refinement by operating over directed rather than undirected graphs. Our approach also draws inspiration from DFA partition refinement strategies, such as Hopcraft's DFA minimization algorithm [8, 6].

We now define the following stabilization condition which will tell us when to stop computing new colors:

Global Stabilization Condition: Global stabilization is achieved in round i if for all nodes u, v in G , $\lambda_{i+1}(u) = \lambda_{i+1}(v)$ if and only if $\lambda_i(u) = \lambda_i(v)$.

Our first step of local graph hashing, then, is to take the input graph and compute the stable colors of each node, stopping when the global stabilization condition is achieved. With this in hand, we then construct a

Algorithm 4.1 Local Digraph Node Hash

```
1: function SERIALIZE( $Q = (V, E, L)$ , root, reach, colors)
2:   order  $\leftarrow$  SORTED(reach, key=colors)
3:   index  $\leftarrow \{x \mapsto i \mid (i, x) \in \text{ENUMERATE}(\text{order})\}$ 
4:   root_idx  $\leftarrow$  index[root]
5:   adj_list = SORTED([(index[s], index[t]  $\mid (s, t) \in E \wedge s \in \text{reach} \wedge t \in \text{reach}$ )])
6:   labels  $\leftarrow [L(x) \text{ for } x \text{ in order}]$ 
7:   return  $\mathcal{H}(\text{TO\_STR}((\text{root\_idx}, \text{labels}, \text{adj\_list})))$ 
8: end function
9: function  $\mathcal{H}_{\mathcal{L}}(G = (V, E, L))$ 
10:    $C \leftarrow \text{GLOBAL\_COLORS}(G)$  ▷ Map nodes in  $V$  to colors
11:    $Q \leftarrow G/C$  ▷ Use colors as an equivalence class to compute a quotient graph
12:    $(V', E', L') \leftarrow Q$  ▷  $L'$  maps nodes in  $V'$  to the original node labels
13:   reachable  $\leftarrow \{n \mapsto \text{REACHABLE}(Q, n) \mid n \in V'\}$ 
14:   stable  $\leftarrow \{\}$ 
15:   final_hash  $\leftarrow \text{EMPTY\_DICT}()$ 
16:   colors  $\leftarrow \{n \mapsto \mathcal{H}(L'(n)) \mid n \in V'\}$ 
17:   while  $|\text{stable}| < |V'|$  do
18:     new_colors  $\leftarrow \text{REFINE\_ROUND}(Q, \text{colors}, L')$ 
19:     for  $n \in V'$  do
20:       if  $n \notin \text{stable}$  then
21:         if PARTITION_UNCHANGED(reachable[n], colors, new_colors) then
22:           stable  $\leftarrow \text{stable} \cup \{n\}$ 
23:           final_hash[n]  $\leftarrow \text{SERIALIZE}(Q, n, \text{reachable}[n], \text{colors})$ 
24:         end if
25:       end if
26:     end for
27:   end while
28:   return  $\{n \mapsto \text{final\_hash}[C[n]] \mid n \in V\}$ 
29: end function
```

quotient graph induced by the stable colors, merging any nodes together with identical color. Note here that the quotient graph produced is a multi-digraph, so that we preserve the unfolding of all nodes.

We then proceed to operate on the new quotient graph, and pass it through a fresh round of color refinement, this time making use of the following stabilization condition:

Local Stabilization Condition: A node n is locally stable in round i if for all nodes u, v in the reachable subgraph from n , $\lambda_{i+1}(u) = \lambda_{i+1}(v)$ if and only if $\lambda_i(u) = \lambda_i(v)$.

For each node n in the quotient graph, there will be some step i in which that node becomes locally stable. On that round we can create a canonical order for the nodes in the reachable subgraph of n by sorting the colors λ_i for each node. We can then use this ordering to create a canonical adjacency list, which can then be serialized to a string and passed through a string hashing function. This gives the node hash for vertex n .

To summarize our approach, we begin by merging any nodes which are identical modulo tree expansion by using graph coloring with the global stabilization condition. We then construct a quotient graph, merging any of these identical nodes. We then compute a new graph coloring for each node, this time using the local stabilization condition. Since these colors are unique, they can be canonically ordered. The canonical ordering can then be used to construct a canonical adjacency list, which can then be hashed using an ordinary string hashing function. Algorithm 4.1 gives a pseudo-code implementation of this approach that computes the hash for all nodes in the input graph.

THEOREM 4.1 (Local Hash Proof of Correctness). *Given some node u from G_1 and some node v from G_2 ,*

$\mathcal{H}_{\mathcal{L}}(G_1)[u] = \mathcal{H}_{\mathcal{L}}(G_2)[v]$ if and only if $\text{TREE_EXPANSION}(G_1, u) \cong \text{TREE_EXPANSION}(G_2, v)$

Proof. ($\Rightarrow$) If $\mathcal{H}_{\mathcal{L}}(G_1)[u] = \mathcal{H}_{\mathcal{L}}(G_2)[v]$, then the reachable subgraphs in the quotient graph from u and v are identical. Since the tree expansion depends only on the reachable parts of the graph, and taking the quotient leaves the tree expansion unchanged, the tree expansions are isomorphic.

($\Leftarrow$) Let V_1, E_1 be the vertices and edges of the quotient graph $Q_1 = G_1/C_1$ reachable from u . Let V_2, E_2 be the vertices and edges of the quotient graph $Q_2 = G_2/C_2$ reachable from v .

Then it is easy to construct a bijection f of the vertices in the tree expansions since they are isomorphic. This bijection maps between nodes with isomorphic subtrees. Since the quotient graph merges any nodes with identical tree expansions, this same bijection can be re-interpreted as a bijective induced map $g : V_1 \rightarrow V_2$ between the nodes in the reachable portions of the quotient graphs.

We now show that g also preserves edges. That is, $(a, b) \in E_1$ if and only if $(g(a), g(b)) \in E_2$.

Consider the following cases:

- If $(a, b) \in E_1$, then $(g(a), g(b)) \in E_2$. Assume that $(g(a), g(b)) \notin E_2$. Then $\text{TREE_EXPANSION}(G_1, a) \not\cong \text{TREE_EXPANSION}(G_2, g(a))$, which contradicts the assumption that the original bijection f maps between isomorphic tree expansions.
- If $(g(a), g(b)) \in E_2$, then $(a, b) \in E_1$. Assume that $(a, b) \notin E_1$. Then $\text{TREE_EXPANSION}(G_1, a) \not\cong \text{TREE_EXPANSION}(G_2, g(a))$, which contradicts the assumption that the original bijection f maps between isomorphic tree expansions.

We have therefore shown that the reachable portions of the quotient graph are isomorphic. We now note that the process of computing the final_hash values in Algorithm 4.1 is permutation invariant with respect to the quotient graph. This is because the color refinement portion of the computation is independent of vertex order, and the locally stable colors for each node when PARTITION_UNCHANGED is true must be unique (by virtue of operating over a globally stable color quotient graph). We can therefore conclude that $\mathcal{H}_{\mathcal{L}}(G_1)[u] = \mathcal{H}_{\mathcal{L}}(G_2)[v]$. $\square$

4.1 Alternative Formulations of Local Hashing The strategy employed for local hashing depends on properties we can observe by traversing the graph forward along its directed edges. There are other formulations of local hashing, two of which we will note here:

1. As a *directed vertex language*. In this formulation, the hash of a node is defined as the hash of the (potentially infinite) set of strings that can be formed by visiting labeled nodes:

$$(4.3) \quad \text{Lang}_G(u) = \{L(u)\} \cup \bigcup_{v \in \text{successor}(u)} \{L(u) \circ x \mid x \in \text{Lang}_G(v)\}$$

This particular formulation is very similar to that of nondeterministic finite automata, except that characters are accumulated in a string by visiting nodes (instead of traversing edges), and all states can be terminal states.

Our primary note on this formulation of local graph hashing is that computing a hash value is **PSPACE-hard**. For this reason, we will not elaborate much further on solving this problem as an efficient algorithm is not possible.

Proof Sketch: We perform a poly-time reduction from the NFA universality problem (UNIV), which is **PSPACE-complete** [12]. We say that an NFA is universal if it accepts every string. Given an input NFA, we can perform a poly-time version to an equivalent directed vertex language. We can then construct a universal NFA and encode it in a directed vertex language. If we could compute the hash of a directed vertex language, we could decide the equality between the input NFA and the universal NFA.

2. As a *bisimulation system*. The approach given in Section 4 seeks to hash the unfolding, which is a slight twist on the philosophy of bisimulation. In a typical bisimulation setup, we say that nodes a and b are bisimilar if they have the same labels and if the following two conditions are met:

- For every outgoing edge $a \rightarrow a'$, there exists a $b \rightarrow b'$ such that a' and b' are bisimilar.

- For every outgoing edge $b \rightarrow b'$, there exists a $a \rightarrow a'$ such that b' and a' are bisimilar.

Note the use of the existential operator instead of a bijective mapping between successor nodes. This means that bisimulation inherently ignores duplicate bisimilar states. For example, two nodes may be bisimilar but have different out-degrees if multiple successor nodes are themselves bisimilar. Extending computation of a node's hash modulo bisimilarity instead of tree expansion requires only a small modification to the previously described procedure: when computing the quotient graph G/C , collapse the graph to an ordinary digraph instead of a multi-digraph.

5 Outgoing Ordered Edge Graphs For many real world applications of graph hashing, the labels of outgoing edges are strictly ordered. We can take advantage of this extra structure on the edges to create simpler hashing algorithms. The algorithms in this section were directly inspired by the more general Scott graph canonization algorithm given by Bloyet, Marteau and Frénod [4]. The basic idea for our approach is that due to the ordering of the edges, we can give an order to the nodes based on when they are visited in a graph traversal algorithm (in this case depth-first search).

Algorithm 5.1 Node Hash - Outgoing Ordered Edges

1. For some input node n , perform a depth first search starting at n and record the visitation order of the reachable nodes. Relabel each node in the order visited by the DFS.
 2. Create an adjacency list of this relabeled subgraph. Sort this adjacency list.
 3. Create a list of the original labels in the visitation order.
 4. Use a string hashing $\mathcal{H}$ on the (label list, adjacency list) tuple. This gives the hash of n .
-

For the **special case of a strongly connected graph**, we can use these hashes to compute both a canonical order and orbit partition in quadratic time. This special case is useful for the Merkle-style graph hashing algorithm given in Section 6 where strongly connected components are hashed recursively.

Algorithm 5.2 Canonization and Orbit Partitioning for Strongly Connected Outgoing Ordered Edge Graphs

1. For each node n , compute the hash using Algorithm 5.1. Nodes in the same orbit will have identical hash values, which can be used to compute the orbit partition. **Proof:** It is easy to construct an automorphism by examining the adjacency list constructed during the DFS.
 2. Take the node with the minimal hash value, and use its DFS ordering as the canonical order for the graph.
 3. At this point we now have a canonical order and orbit partition that can be used in Algorithm 3.1 or 3.2.
-

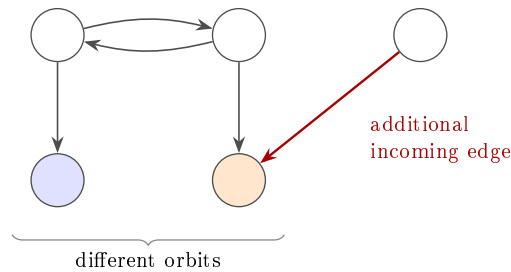


Figure 5.1: The symmetry of the two bottom nodes is broken by considering incoming edges from higher in the graph. Without the highlighted edge the two bottom nodes would be interchangeable; with it, no automorphism maps one onto the other, so the two bottom nodes no longer lie in the same orbit.

Why must we restrict ourselves to the special case of strongly connected graphs? In the case of a generic digraph, the symmetry and therefore the orbit of a node depends not only on the structure of the reachable subgraph, but also on the structure from ancestor nodes. Incoming edges from ancestor nodes can break the symmetry of an orbit, as demonstrated in Figure 5.1. In the case of a strongly connected graph, this is not an issue as all nodes are mutual descendants.

6 Merkle Graph Hashing For applications where graphs are built up over time taking a recursive algorithmic approach similar to Merkle-tree hashing would be highly beneficial. We will now construct such an algorithm that recursively operates over the condensation graph, which is a directed acyclic graph by construction. Algorithm 6.1 gives a pseudo-code implementation of the approach we will now discuss.

To compute the hash of a specific node, we begin by computing the strongly connected component containing the node. We then recursively compute the hashes of the targets of any edges leaving the strongly connected component. New labels for each node in the SCC are created by concatenating on the multiset of the hashes of non-SCC targets. Finally we compute the hash of each node using Algorithm 3.2.

Algorithm 6.1 Merkle Graph Node Hashing

```

1: function  $\mathcal{H}_{\mathcal{NM}}(v, G=(V, E, L))$ 
2:    $S \leftarrow \text{SCC}(v, G)$   $\triangleright S$  is the set of vertices in the SCC containing  $v$ 
3:    $L'(u) = \text{TO\_STR}((L(u), \{\{\mathcal{H}_{\mathcal{NM}}(w, G) \mid u \in S \wedge w \notin S \wedge (u, w) \in E\}\}))$ 
4:    $\triangleright$  Relabel all nodes in the SCC using the recursively computed hashes
5:    $(V', E', L'') \leftarrow \text{SUBGRAPH}(S, G)$ 
6:    $G' \leftarrow (V', E', L')$ 
7:   return  $\mathcal{H}_{\mathcal{N}}(v, G')$   $\triangleright$  Hash the relabelled SCC containing  $v$ 
8: end function

```

7 Application Sketch From the algorithms given so far, it may not be obvious how to implement these graph theoretic procedures for real world data. Here we will discuss how to do so by taking the example of a static hyperlinked web page system stored in a content addressed distributed hash table (DHT).

In this sketch, we will assume that our web pages are static HTML files which link to each other and to external websites. These files form a directed graph where each page is a node, and links between pages are edges. The outgoing edges from a node are labeled by natural numbers in the order in which the links appear in the HTML file. Figure 7.1 gives a concrete example of the process described here.

We begin by considering the links within each file and the strongly connected components created by our website. All intra-SCC links are replaced with natural numbers in the order in which the links appear. Links leaving the SCC are recursively hashed, and these hash values are inserted verbatim into the HTML of our webpages. We then compute the canonical order and orbit partition of the SCC in polynomial time using Algorithm 5.2.

We then insert the SCC into the DHT as follows:

- **Key:** The hash of the SCC computed via Algorithm 3.1.
- **Value:** The canonized adjacency list and the label list (which is ordered in the canonical order).

For each page in the SCC, we insert the following into the DHT:

- **Key:** The string hash of the (orbit index, SCC hash). We compute the orbit index as is done in Algorithm 3.2.
- **Value:** The (orbit index, SCC hash) pair.

Suppose that a user wants to navigate our website, and has obtained a hash for one specific page. The browser software will begin by retrieving the orbit index and SCC hash from our distributed hash table (DHT). The browser then retrieves the SCC, giving us the adjacency list and allowing the SCC digraph to be reconstructed. The orbit index will then point us to a specific node in the SCC. The browser will display the contents of this node to the user.

Row 1: Original HTML Pages

index.html	about.html	contact.html
<pre> <body> <h1>Welcome</h1> About Us External Resource </body> </pre>	<pre> <body> <h1>About Us</h1> Home Contact Partner Site </body> </pre>	<pre> <body> <h1>Contact Us</h1> About External Resource </body> </pre>

Row 2: Replace intra-SCC links with natural numbers in order of appearance, recursively hash inter-SCC links

index.html	about.html	contact.html
<pre> <body> <h1>Welcome</h1> About Us External Resource </body> </pre>	<pre> <body> <h1>About Us</h1> Home Contact Partner Site </body> </pre>	<pre> <body> <h1>Contact Us</h1> About External Resource </body> </pre>

Row 3: Construct strongly connected directed graph (labels taken from previous row)

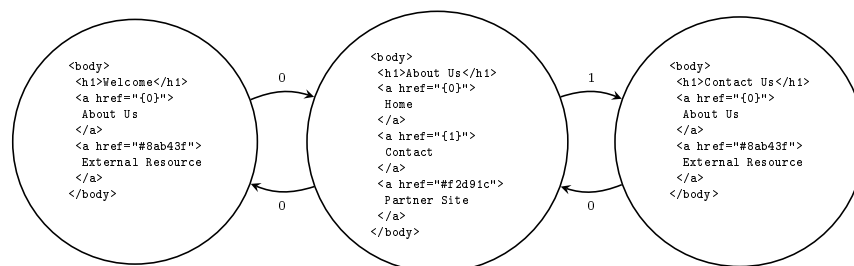


Figure 7.1: Visualization of procedure for hashing a three page HTML website. Note that in Row 2, the values #8ab43f and #f2d91c refer to hashes computed recursively - in this case the hash of pages on external websites.

If the user clicks a link within the same SCC, the browser simply follows the graph edge and shows the appropriate page contents. For links that leave the SCC, the browser repeats the same process from the beginning, retrieving the new SCC from the DHT.

8 Related Work The core of the digraph Merkle hashing algorithm was independently discovered by the developers of the Unison programming language for use in hashing abstract syntax trees of the Unison language. However, their approach does not take into account computation of the orbits of the nodes, and

The Weisfeiler Lehman graph hash [19] is a kernel method that iteratively hashes and aggregates neighborhoods of each node in an input graph. Isomorphic graphs are guaranteed to result in equal hashes, and a strong guarantee is given that non-isomorphic graphs will have different hashes. This kernel method takes an iteration parameter as input, which changes the result of the hashes computed for different settings of this parameter. Our algorithm gives similar guarantees as the Weisfeiler Lehman graph hash, with the added benefit that there is no requirement for an iteration parameter.

In a 2014 paper, Arshad et al. [3] presented an algorithm for hashing directed graphs, given additional inputs of a fixed source node and depth-first search traversal tree. The depth-first search traversal tree breaks the graph up into different edge types (tree, back, forward, and cross edges) depending on search order. The hashing algorithm then uses the traversal numbers to order the nodes and construct the hash. The authors note that isomorphic graphs may hash to different values due to this input dependence on source node and DFS tree. In contrast, the algorithms presented in this paper do not rely on an input DFS tree or node ordering.

In a separate 2018 paper, Arshad et al. [2] presented a graph hashing algorithm where the hash of a node depends on the XORing together the hashes of the labels of all of that node's descendants. The hash of the entire graph is then computed by hashing each node concatenated with the accumulated graph hash. This approach is problematic in two different ways. While the node hash heuristic may be admissible in many cases, it is easy to construct two non-isomorphic graphs containing nodes with equal hashes, and therefore also equal graph hashes. Indeed, consider a k -clique and a k -cycle graph containing nodes with equivalent labels. These two graphs are not isomorphic, but each node has $k - 1$ descendants, each of which are identically labelled. This also shows that the hash of the entire graph are equal. Another problematic aspect of their approach lies with the dependence on DFS order. Two isomorphic graphs may have different graph hashes depending on the DFS order.

9 Experimental Results

References

- [1] *IPFS is the distributed web*, <https://ipfs.io/>. Accessed: 2019-12-09.
- [2] M. U. ARSHAD, A. KUNDU, E. BERTINO, A. GHAFOR, AND C. KUNDU, *Efficient and scalable integrity verification of data and query results for graph databases*, IEEE Trans. Knowl. Data Eng., 30 (2018), pp. 866–879.
- [3] M. U. ARSHAD, A. KUNDU, E. BERTINO, K. MADHAVAN, AND A. GHAFOR, *Security of graph data: hashing schemes and definitions*, in Proceedings of the 4th ACM conference on Data and application security and privacy, CODASPY '14, New York, NY, USA, Mar. 2014, Association for Computing Machinery, pp. 223–234.
- [4] N. BLOYET, P.-F. MARTEAU, AND E. FRÉNOT, *Scott: A method for representing graphs as rooted trees for graph canonization*, in Complex Networks and Their Applications VIII, Studies in computational intelligence, Springer International Publishing, Cham, 2020, pp. 578–590.
- [5] P. T. DARGA, M. H. LIFFITON, K. A. SAKALLAH, AND I. L. MARKOV, *Exploiting structure in symmetry detection for CNF*, in Proceedings of the 41st annual Design Automation Conference, ACM, 2004, pp. 530–534.
- [6] J. HOPCROFT, *An $n \log n$ algorithm for minimizing states in a finite automaton*, in Theory of Machines and Computations, Z. Kohavi and A. Paz, eds., Academic Press, 1971, pp. 189–196, <https://doi.org/https://doi.org/10.1016/B978-0-12-417750-5.50022-1>, <https://www.sciencedirect.com/science/article/pii/B9780124177505500221>.

- [7] T. JUNTILA AND P. KASKI, *Engineering an efficient canonical labeling tool for large and sparse graphs*, in 2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX), SIAM, 2007, pp. 135–149.
- [8] T. KNUUTILA, *Re-describing an algorithm by hopcroft*, Theor. Comput. Sci., 250 (2001), pp. 333–363.
- [9] J. LOELIGER AND M. MCCULLOUGH, *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*, “O’Reilly Media, Inc.”, Aug. 2012.
- [10] B. D. MCKAY, *Practical graph isomorphism*, Department of Computer Science, Vanderbilt University Tennessee, USA, 1981.
- [11] R. C. MERKLE, *A digital signature based on a conventional encryption function*, in Conference on the theory and application of cryptographic techniques, Springer, 1987, pp. 369–378.
- [12] A. R. MEYER AND L. J. STOCKMEYER, *The equivalence problem for regular expressions with squaring requires exponential space*, in 13th Annual Symposium on Switching and Automata Theory (swat 1972), 1972, pp. 125–129, <https://doi.org/10.1109/SWAT.1972.29>.
- [13] S. NAKAMOTO, *Bitcoin: A peer-to-peer electronic cash system*, (2008).
- [14] N. I. OF STANDARDS AND TECHNOLOGY, *Secure hash standard*, Tech. Report Publication 180-2, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2002, <https://csrc.nist.gov/csrc/media/publications/fips/180/2/archive/2002-08-01/documents/fips180-2.pdf>.
- [15] N. I. OF STANDARDS AND TECHNOLOGY, *SHA-3 standard: Permutation-based hash and extendable-output functions*, Tech. Report Publication 202, 100 Bureau Drive, Stop 8900, Gaithersburg, MD 20899-8900, August 2015, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>.
- [16] C. PICARD AND R. MATTHES, *Coinductive graph representation: the problem of embedded lists*, Electronic Communications of the EASST, 39 (2011), <https://doi.org/10.14279/tuj.eceasst.39.649>, <https://eceasst.org/index.php/eceasst/article/view/2664>.
- [17] R. RIVEST, *The MD5 message-digest algorithm*, Tech. Report RFC 1321, MIT Laboratory for Computer Science and RSA Data Security, Inc., 545 Technology Square, Cambridge, MA 02139-1986, April 1992, <https://tools.ietf.org/html/rfc1321>.
- [18] D. SANGIORGI, *Introduction to Bisimulation and Coinduction*, Cambridge University Press, 2011.
- [19] N. SHERVASHIDZE, P. SCHWEITZER, E. J. VAN LEEUWEN, K. MEHLHORN, AND K. M. BORGWARDT, *Weisfeiler-Lehman graph kernels*, J. Mach. Learn. Res., 12 (2011).